

Practical No.9

MOHAMMAD QASIM

13-09-2025

Objective:

Fit the multiple linear regression model $Y = X\beta + \epsilon$ to the following data using matrix algebra. also find the R^2 , $adj. R^2$, σ_e^2 , variance covariance matrix of $\hat{\beta}$, Confidence interval of $\hat{\beta}$, test statistics and p-value corresponding to all predictor.

Python Code

```
In [1]: import numpy as np
y=np.reshape([2.2,1,5.45,2.55,4.85,2.92,3.93,1.08,4.62,2.69],(10,1))
y1=[2.2,1,5.45,2.55,4.85,2.92,3.93,1.08,4.62,2.69]
o=[1,1,1,1,1,1,1,1,1,1]
x1= [2,1,4,3,5,2,3.5,0.5,4.5,2.2]
x2= [3,2,0.5,2.5,3.5,1.5,2,0.7,3,1]
x3= [1.5,0.5,2,1,2.5,1.2,1.8,0.3,2.2,0.8]
```

```
In [2]: Y=np.asmatrix(y)
```

```
In [3]: Y
```

```
Out[3]: matrix([[2.2 ],
                [1.  ],
                [5.45],
                [2.55],
                [4.85],
                [2.92],
                [3.93],
                [1.08],
                [4.62],
                [2.69]])
```

```
In [4]: x1
```

```
Out[4]: [2, 1, 4, 3, 5, 2, 3.5, 0.5, 4.5, 2.2]
```

```
In [5]: x2
```

```
Out[5]: [3, 2, 0.5, 2.5, 3.5, 1.5, 2, 0.7, 3, 1]
```

```
In [6]: x3
```

```
Out[6]: [1.5, 0.5, 2, 1, 2.5, 1.2, 1.8, 0.3, 2.2, 0.8]
```

```
In [7]: X=np.matrix([o,x1,x2,x3])
```

```
In [8]: X
```

```
Out[8]: matrix([[1. , 1. , 1. , 1. , 1. , 1. , 1. , 1. , 1. , 1. ],
                [2. , 1. , 4. , 3. , 5. , 2. , 3.5, 0.5, 4.5, 2.2],
                [3. , 2. , 0.5, 2.5, 3.5, 1.5, 2. , 0.7, 3. , 1. ],
                [1.5, 0.5, 2. , 1. , 2.5, 1.2, 1.8, 0.3, 2.2, 0.8]])
```

```
In [9]: X.T
```

```
Out[9]: matrix([[1. , 2. , 3. , 1.5],
                [1. , 1. , 2. , 0.5],
                [1. , 4. , 0.5, 2. ],
                [1. , 3. , 2.5, 1. ],
                [1. , 5. , 3.5, 2.5],
                [1. , 2. , 1.5, 1.2],
                [1. , 3.5, 2. , 1.8],
                [1. , 0.5, 0.7, 0.3],
                [1. , 4.5, 3. , 2.2],
                [1. , 2.2, 1. , 0.8]])
```

```
In [10]: tran=X@X.T
```

```
In [11]: tran
```

```
Out[11]: matrix([[10. , 27.7 , 19.7 , 13.8 ],
                 [27.7 , 96.59, 61.05, 47.51],
                 [19.7 , 61.05, 48.49, 30.76],
                 [13.8 , 47.51, 30.76, 24.  ]])
```

```
In [12]: inv=np.linalg.inv(tran)
```

```
In [13]: inv
```

```
Out[13]: matrix([[ 0.63667152, -0.08698859, -0.13985037, -0.0146434 ],
                 [-0.08698859,  0.40601407,  0.01223824, -0.76940559],
                 [-0.13985037,  0.01223824,  0.14114211, -0.12470979],
                 [-0.0146434 , -0.76940559, -0.12470979,  1.73302548]])
```

```
In [14]: xty=X@Y
```

```
In [15]: xty
```

```
Out[15]: matrix([[ 31.29 ],
                 [105.943],
                 [ 64.221],
                 [ 52.593]])
```

```
In [16]: beta=inv@xty
```

```
In [17]: beta
```

```
Out[17]: matrix([[ 0.95414853],
                 [ 0.6130794 ],
                 [-0.5739368 ],
                 [ 1.16469349]])
```

```
In [18]: y_cap=X.T@beta
```

```
In [19]: y_cap
```

```
Out[19]: matrix([[2.20553718],
                 [1.00170108],
                 [5.44888472],
                 [2.52323823],
                 [4.92250048],
                 [2.71703433],
                 [4.04850113],
                 [1.20834052],
                 [4.55352113],
                 [2.66074121]])
```

```
In [20]: Y-y_cap
```

```
Out[20]: matrix([[ -0.00553718],
                 [ -0.00170108],
                 [  0.00111528],
                 [  0.02676177],
                 [-0.07250048],
                 [  0.20296567],
                 [-0.11850113],
                 [-0.12834052],
                 [  0.06647887],
                 [  0.02925879]])
```

```
In [21]: sse=(Y-y_cap).T@(Y-y_cap)
```

```
In [22]: sse
```

```
Out[22]: matrix([[0.0829917]])
```

```
In [23]: sigma=sse/7
```

```
In [24]: sigma
```

```
Out[24]: matrix([[0.01185596]])
```

```
In [25]: v=0.01185596*inv
```

```
In [26]: v
```

```
Out[26]: matrix([[ 0.00754835, -0.00103133, -0.00165806, -0.00017361],
                 [-0.00103133,  0.00481369,  0.0001451 , -0.00912204],
                 [-0.00165806,  0.0001451 ,  0.00167338, -0.00147855],
                 [-0.00017361, -0.00912204, -0.00147855,  0.02054668]])
```

```
In [27]: se=np.sqrt(np.diag(v))
```

```
In [28]: se
```

```
Out[28]: array([0.08688125, 0.06938074, 0.04090691, 0.14334113])
```

Find R^2

```
In [29]: np.mean(Y)
```

```
Out[29]: np.float64(3.129)
```

```
In [30]: a=Y-3.129
```

```
In [31]: a
```

```
Out[31]: matrix([[ -0.929],
                 [-2.129],
                 [  2.321],
                 [-0.579],
                 [  1.721],
                 [-0.209],
                 [  0.801],
                 [-2.049],
                 [  1.491],
                 [-0.439]])
```

```
In [32]: tss=a.T@a
```

```
In [33]: tss
```

```
Out[33]: matrix([[21.37929]])
```

```
In [34]: R2=1-(sse/tss)
```

```
In [35]: R2
```

```
Out[35]: matrix([[0.99611813]])
```

```
In [36]: import pandas as pd
```

```
In [37]: data=pd.DataFrame({"y":y1,"o":o,"x1":x1,"x2":x2,"x3":x3})
```

```
In [38]: data
```

```
Out[38]:
```

	y	o	x1	x2	x3
0	2.20	1	2.0	3.0	1.5
1	1.00	1	1.0	2.0	0.5
2	5.45	1	4.0	0.5	2.0
3	2.55	1	3.0	2.5	1.0
4	4.85	1	5.0	3.5	2.5
5	2.92	1	2.0	1.5	1.2
6	3.93	1	3.5	2.0	1.8
7	1.08	1	0.5	0.7	0.3
8	4.62	1	4.5	3.0	2.2
9	2.69	1	2.2	1.0	0.8

```
In [39]: y2=np.asmatrix(y1).T
```

```
In [40]: y2
```

```
Out[40]: matrix([[2.2 ],
                 [1.  ],
                 [5.45],
                 [2.55],
                 [4.85],
                 [2.92],
                 [3.93],
                 [1.08],
                 [4.62],
                 [2.69]])
```

```
In [41]: o1=np.asmatrix(o).T
```

```
In [42]: x11=np.asmatrix(x1).T
```

```
In [43]: x22=np.asmatrix(x2).T
```

```
In [44]: x33=np.asmatrix(x3).T
```

```
In [45]: beta
```

```
Out[45]: matrix([[ 0.95414853],
                 [ 0.6130794 ],
                 [-0.5739368 ],
                 [ 1.16469349]])
```

```
In [46]: adj_r=1-(sse/7)/(tss/9)
```

```
In [47]: adj_r
```

```
Out[47]: matrix([[0.99500902]])
```

Results

```
In [48]: R2
```

```
Out[48]: matrix([[0.99611813]])
```

```
In [49]: adj_r
```

```
Out[49]: matrix([[0.99500902]])
```

```
In [50]: sse
```

```
Out[50]: matrix([[0.0829917]])
```

```
In [51]: beta
```

```
Out[51]: matrix([[ 0.95414853],  
                 [ 0.6130794 ],  
                 [-0.5739368 ],  
                 [ 1.16469349]])
```

$$\text{Var}(\hat{\beta}) = \sigma^2(X'X)^{-1}.$$

```
In [52]: v
```

```
Out[52]: matrix([[ 0.00754835, -0.00103133, -0.00165806, -0.00017361],  
                 [-0.00103133,  0.00481369,  0.0001451 , -0.00912204],  
                 [-0.00165806,  0.0001451 ,  0.00167338, -0.00147855],  
                 [-0.00017361, -0.00912204, -0.00147855,  0.02054668]])
```

```
In [53]: se
```

```
Out[53]: array([0.08688125, 0.06938074, 0.04090691, 0.14334113])
```

Confidence Interval ($\hat{\beta}_j \pm t_{\alpha/2} \times SE(\hat{\beta}_j)$)

For ($\hat{\beta}_0$)

```
In [54]: 0.95414853-2.3646242515927844*0.0868812,0.95414853+2.3646242515927844*0.0868812
```

```
Out[54]: (0.7487071374725169, 1.159589922527483)
```

For ($\hat{\beta}_1$)

```
In [55]: 0.6130794-2.3646242515927844*0.06938074,0.6130794+2.3646242515927844*0.06938074
```

```
Out[55]: (0.4490200196025465, 0.7771387803974537)
```

For ($\hat{\beta}_2$)

```
In [56]: -0.5739368-2.3646242515927844*0.04090691,-0.5739368+2.3646242515927844*0.04090691,
```

```
Out[56]: (-0.6706662714437234, -0.4772073285562766)
```

For ($\hat{\beta}_3$)

```
In [57]: 1.16469349-2.3646242515927844*0.14334113,1.16469349+2.3646242515927844*0.14334113,
```

```
Out[57]: (0.8257455777512861, 1.5036414022487141)
```

test statistics

```
In [58]: beta.flatten()/se
```

```
Out[58]: matrix([[ 10.98221423,  8.83644992, -14.03031454,  8.12532633]])
```

```
In [59]: import scipy.stats as st  
st.t.ppf(0.025,df=7,loc=0,scale=1)
```

```
Out[59]: np.float64(-2.3646242515927844)
```

```
In [60]: st.t.ppf(0.975,df=7,loc=0,scale=1)
```

```
Out[60]: np.float64(2.3646242515927844)
```

Interpretation:

Here we fit the multiple linear regression model $Y = X\beta + \epsilon$ of the following data using matrix algebra. also find the $R^2 = 99.611813\%$, $adj. R^2 = 99.500902\%$, $\sigma_e^2 = 0.0829917$, variance covariance matrix of $\hat{\beta}$, Confidence interval of $\hat{\beta}$, test statistics and p-value corresponding to all predictor.